
Absicherung der Inhaltsdaten im Wasserstoffmarkt mittels JSON Web Signature (JWS)

Version 0.9

Inhaltsverzeichnis

1.	Einleitung.....	5
1.1	Ziel des Dokuments	5
1.2	Geltungsbereich und Abgrenzung	5
1.3	Schlüsselwörter	6
1.4	Glossar	6
2.	Signaturverfahren im Überblick	7
2.1	Grundprinzip	7
2.2	Zu sichernde Nachrichten.....	7
3.	Kryptographische Vorgaben	9
3.1	Algorithmen.....	9
3.2	Schlüssel und Zertifikate	10
3.3	Hashverfahren	11
4.	Gemeinsame Signaturmechanik	12
4.1	Serialisierung.....	12
4.2	JOSE-Header	12
4.3	Kanonisierung und Signing Input	13
5.	Signatur im HTTP-Kontext	14
5.1	Bestandteile der Signaturrepräsentation	14
5.2	Normalisierung der Request-URL	16
5.3	Übertragung in HTTP-Headern	17
6.	Signatur im WebSocket-Kontext	19
6.1	Der Envelope als Signaturrepräsentation	19
6.2	Signaturzertifikat der WebSocket-Verbindung	20
6.3	Übertragung der Signatur	21
7.	Signaturerstellung und Signaturprüfung.....	22
7.1	Signaturerstellung	22
7.2	Signaturprüfung.....	22
8.	Replay-Schutz und Zeitfenster	24
8.1	Zeitfenster.....	24
8.2	Einmaligkeit	24
9.	Fehlerbehandlung.....	27
9.1	HTTP-Kontext	27
9.2	WebSocket-Kontext.....	27
10.	Beispiele.....	29

10.1	Beispiel: HTTP-Request mit Payload	29
10.2	Beispiel: signaturpflichtige WebSocket-Nachricht	29
11.	Betriebsparameter.....	31
12.	Quellen	33

Abkürzungsverzeichnis

Abkürzung	Bedeutung
API	Application Programming Interface / Anwendungsschnittstelle
BDEW	Bundesverband der Energie- und Wasserwirtschaft
BOM	Byte Order Mark
BSI	Bundesamt für Sicherheit in der Informationstechnik
CRL	Certificate Revocation List / Zertifikatssperrliste
DER	Distinguished Encoding Rules
ECDSA	Elliptic Curve Digital Signature Algorithm
EMT	Externer Marktteilnehmer (Rolle innerhalb der SM-PKI)
H2-	Namespace-Präfix der in der API-Guideline definierten proprietären HTTP-Header
HTTP	Hypertext Transfer Protocol
I-JSON	Internet JSON (restriktives JSON-Profil gemäß RFC 7493)
JCS	JSON Canonicalization Scheme (RFC 8785)
JOSE	JavaScript Object Signing and Encryption
JSON	JavaScript Object Notation
JWS	JSON Web Signature (RFC 7515)
MP-ID	Marktpartner-ID
mTLS	Mutual Transport Layer Security
OU	Organizational Unit (Feld im Zertifikat)
RFC	Request for Comments
SHA	Secure Hash Algorithm
SM-PKI	Smart Meter Public Key Infrastructure
TLS	Transport Layer Security
URI	Uniform Resource Identifier
UTC	Coordinated Universal Time / koordinierte Weltzeit
UTF	Unicode Transformation Format
UUID	Universally Unique Identifier
WS	WebSocket

1. Einleitung

1.1 Ziel des Dokuments

Dieses Dokument (JWS-SPEC) ist die Technische Spezifikation zur Inhaltsdatensicherung auf Basis von JSON Web Signature (JWS) für API-Webdienste im Wasserstoffmarkt. Es konkretisiert:

- die Inhaltsdatensicherungsebene für HTTP-Requests und -Responses gemäß Teil 2, Kapitel 2.3 der API-Guideline Wasserstoffmarkt [API-GL], und
- die Nachrichtensignatur innerhalb der WebSocket-Verbindung des Verzeichnisdienstes gemäß Teil 2, Kapitel 5.2 [API-GL] und der Technischen Schnittstellenbeschreibung [WS-SPEC], Kapitel 3.5.

Beide Anwendungsfälle nutzen dasselbe kryptographische Verfahren (Kapitel 3) und dieselbe Signaturtechnik (JWS, Detached Compact Serialization, JCS-Kanonisierung). Sie unterscheiden sich in der Bildung der zu signierenden Repräsentation und in der Übertragung der Signatur; dies wird in den Kapiteln 5 (HTTP-Kontext) und 6 (WebSocket-Kontext) getrennt festgelegt.

Die in der API-Guideline Wasserstoffmarkt und in [WS-SPEC] getroffenen Regelungen gelten unverändert fort. Dieses Dokument konkretisiert diese Regelungen und wiederholt sie nur, soweit dies für das Verständnis erforderlich ist. Bei Widersprüchen gehen die Regelungen der API-Guideline vor.

1.2 Geltungsbereich und Abgrenzung

Diese Spezifikation gilt für:

- alle fachlichen HTTP-Requests und Retry-Requests sowie asynchronen fachlichen Responses im Geltungsbereich der API-Guideline Wasserstoffmarkt,
- fachlich relevante synchrone HTTP-Responses im Geltungsbereich der API-Guideline Wasserstoffmarkt, das heißt Responses, die einen Verzeichniseintrag oder einen anderen signierten Datenbestand zurückgeben oder eine fachliche Zustandsänderung verbindlich bestätigen oder ablehnen (einschließlich fachlicher Ablehnungen mit ErrorResponse-Body), und
- signaturpflichtige Nachrichten innerhalb der WebSocket-Verbindung des Verzeichnisdienstes gemäß [WS-SPEC], Kapitel 3.5 (subscribe, unsubscribe, availabilityStart, availabilityHeartbeat, availabilityStop).

Die Absicherung der Transportebene (HTTPS bzw. WSS mit mTLS gemäß Teil 2, Kapitel 2.2 [API-GL]) bleibt unberührt und wird durch diese Spezifikation vorausgesetzt. Die Signatur auf Inhaltsdatensicherungsebene ergänzt die Transportsicherung; die übertragenen Nutzdaten werden nicht zusätzlich verschlüsselt.

1.3 Schlüsselwörter

Die Schlüsselwörter „MUSS“/„MÜSSEN“ (**MUST**), „DARF NICHT“ (**MUST NOT**), „ERFORDERLICH“ (**REQUIRED**), „SOLLTE“ (**SHOULD**), „SOLLTE NICHT“ (**SHOULD NOT**), „EMPFOHLEN“ (**RECOMMENDED**), „NICHT EMPFOHLEN“ (**NOT RECOMMENDED**), „KANN/DARF“ (**MAY**) und „OPTIONAL“ (**OPTIONAL**) sind gemäß [RFC2119] und [RFC8174] zu interpretieren, wenn sie in Großbuchstaben verwendet werden.

1.4 Glossar

Begriff	Beschreibung
Ersteller	System, das eine Nachricht für die aktuelle Kommunikationsstrecke erzeugt und signiert. Bei HTTP-Requests ist dies der mTLS-Client der aktuellen Strecke; bei synchronen HTTP-Responses der API-Anbieter der aktuellen Strecke; bei signaturpflichtigen WebSocket-Nachrichten der authentifizierte Client.
Prüfer	System, das eine empfangene Nachricht und deren Signatur verifiziert.
JOSE-Header	Geschützter Header (Protected Header) der JWS mit Algorithmus-, Schlüssel- und Typangaben.
Detached Payload	Betriebsart der JWS, bei der der Payload nicht Bestandteil der Compact Serialization ist, sondern getrennt zur Signaturprüfung herangezogen wird ([RFC7515], Anhang F; [RFC7797]).
Kanonisierung	Überführung eines JSON-Dokuments in eine eindeutige, byteidentisch reproduzierbare Darstellung gemäß JCS [RFC8785].
Signaturrepräsentation	Das JSON-Objekt, dessen JCS-kanonisierte Form den Signing Input bildet: im HTTP-Kontext das Objekt aus meta und data (Kapitel 5.1), im WebSocket-Kontext das Objekt aus connectionId und message, wobei message dem Envelope ohne die Eigenschaft signature entspricht (Kapitel 6.1).
Signing Input	Oktettfolge, über die die Signatur berechnet wird (Kapitel 4.3).
kid	Key-ID: Kennung des Signaturschlüssels im JOSE-Header (Kapitel 3.2).
Technischer Signierer	System, dessen EMT-API-Zertifikat die Nachricht der aktuellen Kommunikationsstrecke signiert. Bei einem direkten HTTP-Request wird es durch H2-Message-Sender, bei einer Data-Hub-Weiterleitung durch H2-Forwarded-By bezeichnet.
connectionId	Bindungskennung einer WebSocket-Verbindung. Sie entspricht der H2-Transaction-Id des erfolgreichen HTTP-Upgrade-Requests und ist Bestandteil der WebSocket-Signaturrepräsentation, wird aber nicht im Envelope übertragen.

2. Signaturverfahren im Überblick

2.1 Grundprinzip

Jede zu sichernde Nachricht wird mit einer JWS in Compact Serialization mit Detached Payload signiert ([RFC7515], Anhang F). Der Payload wird nicht Base64url-kodiert (`b64 = false` gemäß [RFC7797]), sondern in seiner JCS-kanonisierten Form ([RFC8785]) unmittelbar in den Signing Input einbezogen.

Im HTTP-Kontext wird die Signatur in den HTTP-Headern `H2-Signature`, `H2-Digest` und `H2-Cert` übertragen (Kapitel 5.3). Dieses Verfahren entspricht funktional der Inhaltsdatensicherung der BDEW-Regelungen [RzÜ-API] mit den dortigen Headern `X-BDEW-SIGNATURE`, `X-BDEW-DIGEST` und `X-BDEW-CERT`; anstelle des `X`-Präfix wird gemäß Teil 1, Kapitel 4.5.1 [API-GL] durchgängig das Präfix `H2-` verwendet.

Im WebSocket-Kontext gibt es keine HTTP-Header je Nachricht. Die Signatur wird stattdessen in der Envelope-Eigenschaft `signature` der signaturpflichtigen Nachricht selbst übertragen (Kapitel 6.3). Das SIG-Zertifikat wird einmalig im HTTP-Upgrade-Request über `H2-Cert` übertragen, geprüft und an den Verbindungskontext gebunden (Kapitel 6.2).

2.2 Zu sichernde Nachrichten

MUST Im HTTP-Kontext MÜSSEN alle fachlichen HTTP-Requests im Sinne der API-Guideline signiert werden, unabhängig davon, ob sie einen Payload enthalten. Synchrone HTTP-Responses MÜSSEN signiert werden, sofern sie fachlich relevant sind (Teil 2, Kapitel 2.3 [API-GL]); dies gilt auch für fachliche Ablehnungen mit `ErrorResponse-Body`. Rein technische oder transportbezogene Responses sowie Responses ohne Body werden NICHT signiert.

MUST NOT Der HTTP-Upgrade-Request zum Aufbau einer WebSocket-Verbindung und sämtliche zu diesem Upgrade gehörenden HTTP-Responses sind von der Signaturpflicht ausgenommen. Ihre Absicherung erfolgt durch die mTLS-Authentisierung und die Prüfungen gemäß [WS-SPEC].

MUST Die Signaturpflicht für synchrone Responses gilt für applikativ durch den API-Webdienst erzeugte Responses. Vorgelagerte Infrastrukturkomponenten ohne Zugriff auf den Signaturschlüssel des API-Anbieters (z. B. Load Balancer, Gateways, Web Application Firewalls) DÜRFEN negative Responses NICHT mit einem `JSON-Body` im Sinne des `ErrorResponse-Schemas` erzeugen; solche Responses MÜSSEN ohne Body oder mit einem `Content-Type` ungleich `application/json` erfolgen und werden NICHT signiert.

Solche Infrastruktur-Responses enthalten keinen `ErrorResponse-Body` und damit insbesondere kein Feld `transactionId`. Ihre Korrelation zu einem Request ist ausschließlich über die mitgeführten HTTP-Header, insbesondere `H2-Reference-Id` oder `H2-Transaction-Id`, möglich, soweit diese von der Infrastrukturkomponente übernommen werden. Ein Client DARF das Vorhandensein eines `transactionId`-Feldes bei solchen Responses NICHT voraussetzen.

MUST Clients MÜSSEN unsigned negative Responses ohne ErrorResponse-JSON-Body als rein transportbezogene Fehler behandeln; eine fachliche Auswertung DARF NICHT erfolgen.

MUST NOT Aus einer unsigned Response DARF der Client keine fachliche Endgültigkeit ableiten. Insbesondere DARF ein unsigned Fehler NICHT als endgültige fachliche Ablehnung, Bestätigung oder Zustandsänderung gewertet werden; der zugrunde liegende Vorgang gilt als technisch fehlgeschlagen und offen und ist gemäß den festgelegten Retry-Regeln zu behandeln.

MUST Im WebSocket-Kontext MÜSSEN die Client-Nachrichten `subscribe`, `unsubscribe`, `availabilityStart`, `availabilityHeartbeat` und `availabilityStop` signiert werden ([WS-SPEC], Kapitel 3.5). Server-Nachrichten werden NICHT auf Nachrichtenebene signiert.

MUST NOT Unsigned Nachrichten, für die eine Signaturpflicht nach diesem Kapitel besteht, DÜRFEN vom Prüfer NICHT fachlich verarbeitet werden. Sie MÜSSEN gemäß Kapitel 9 abgelehnt werden.

3. Kryptographische Vorgaben

3.1 Algorithmen

MUST Als Signaturverfahren MUSS ECDSA gemäß den in [TR03116-3], Kapitel 8 und 9, beschriebenen kryptographischen Verfahren und Umsetzungshinweisen verwendet werden (Teil 2, Kapitel 2.3 [API-GL]). Im JOSE-Header sind in diesem Profil ausschließlich die beiden folgenden, durch die API-Governance freigegebenen Algorithmen zulässig: `urn:h2:jws:alg:brainpoolP256r1-sha256` (ECDSA mit der Kurve `brainpoolP256r1` und SHA-256) und `urn:h2:jws:alg:brainpoolP384r1-sha384` (ECDSA mit der Kurve `brainpoolP384r1` und SHA-384). Diese beiden Bezeichner bilden das für die Signaturzertifikate der SM-PKI festgelegte JWS-Algorithmusprofil.

MUST Der Bezeichner `urn:h2:jws:alg:brainpoolP256r1-sha256` DARF ausschließlich mit einem SIG-Zertifikat auf der Kurve `brainpoolP256r1` (Objektkennung 1.3.36.3.3.2.8.1.1.7) und `urn:h2:jws:alg:brainpoolP384r1-sha384` ausschließlich mit einem SIG-Zertifikat auf der Kurve `brainpoolP384r1` (Objektkennung 1.3.36.3.3.2.8.1.1.11) verwendet werden. Der Prüfer MUSS die `namedCurve` des öffentlichen Schlüssels des SIG-Zertifikats gegen `alg` prüfen und bei Abweichung ablehnen.

MUST NOT Ein SIG-Zertifikat mit einer anderen Kurve DARF NICHT unter diesen Bezeichnern verwendet werden; dies schließt insbesondere die NIST-Kurven P-256 und P-384 sowie andere Brainpool-Kurvenstärken ein. Die JOSE-Bezeichner `ES256` und `ES384` DÜRFEN in diesem Profil NICHT verwendet werden. Der Prüfer MUSS eine feste Positivliste anwenden und ausschließlich die beiden Bezeichner `urn:h2:jws:alg:brainpoolP256r1-sha256` und `urn:h2:jws:alg:brainpoolP384r1-sha384` akzeptieren; jeder andere `alg`-Wert MUSS abgelehnt werden.

MUST NOT Der Algorithmus `none` sowie RSA- und HMAC-basierte Algorithmen DÜRFEN NICHT verwendet werden. Der Prüfer DARF den anzuwendenden Algorithmus NICHT aus dem `alg`-Parameter allein übernehmen, sondern MUSS ihn gegen die Liste der zulässigen Algorithmen prüfen (Schutz vor Algorithmus-Verwechslung).

Der Signaturwert wird gemäß [RFC7515], Anhang A.3, als Konkatenation `R || S` mit fester Länge kodiert (64 Oktette bei `urn:h2:jws:alg:brainpoolP256r1-sha256`, 96 Oktette bei `urn:h2:jws:alg:brainpoolP384r1-sha384`). Dies entspricht dem Plain Format nach [TR03111]; eine ASN.1-/DER-Kodierung des Signaturwerts DARF NICHT verwendet werden.

Anmerkung: RFC 9864 definiert `ESB256` und `ESB384` als vollständig spezifizierte ECDSA-Algorithmuskennungen für COSE. Diese Kennungen sind keine registrierten JOSE-`alg`-Werte. Für das in diesem Dokument definierte JWS-Profil gelten daher weiterhin ausschließlich die beiden Bezeichner `urn:h2:jws:alg:brainpoolP256r1-sha256` und `urn:h2:jws:alg:brainpoolP384r1-sha384`. Eine Änderung oder Erweiterung dieser Positivliste bedarf einer

ausdrücklichen Festlegung der API-Governance. Da verbreitete JOSE-Bibliotheken Brainpool-Kurven häufig nicht unmittelbar unterstützen, ist der Signatur- und Prüfpfad über eine Brainpool-fähige Kryptobibliothek oder einen entsprechenden Kryptografieprovider bereitzustellen.

MUST Weitere Vertrauensdomänen sowie die ihnen zugeordneten JWS-Algorithmusprofile — einschließlich der jeweils zulässigen `alg`-Werte, Kurven und Zertifikatsvertrauensanker — werden durch die API-Governance festgelegt. Bis zu einer solchen Festlegung gilt ausschließlich das in diesem Kapitel definierte Profil für Signaturzertifikate der SM-PKI; SIG-Zertifikate anderer Vertrauensdomänen MÜSSEN abgelehnt werden.

3.2 Schlüssel und Zertifikate

MUST Zum Signieren MUSS der private Schlüssel des gültigen SIG-Zertifikats aus einem EMT .API-Zertifikatstripel der SM-PKI verwendet werden. Das Feld Organizational Unit des Subject MUSS die MP-ID des technischen Signierers enthalten.

MUST Im HTTP-Kontext MUSS H2-Cert das verwendete SIG-Zertifikat enthalten. Dieses MUSS dem technischen Signierer der aktuellen Kommunikationsstrecke gemäß Teil 1, Kapitel 2.3.4 [API-GL] zugeordnet sein.

MUST Bei HTTP-Requests MÜSSEN das SIG-Zertifikat und das für mTLS verwendete TLS-Client-Zertifikat demselben gültigen EMT .API-Zertifikatstripel und derselben MP-ID zugeordnet sein.

MUST Bei synchronen HTTP-Responses MÜSSEN das SIG-Zertifikat und das für mTLS verwendete TLS-Server-Zertifikat demselben gültigen EMT .API-Zertifikatstripel und derselben MP-ID zugeordnet sein.

MUST NOT Das SIG-Zertifikat und das TLS-Zertifikat SIND unterschiedliche Zertifikate mit unterschiedlichen Verwendungszwecken und DÜRFEN NICHT gegeneinander ausgetauscht werden.

MUST Im WebSocket-Kontext MUSS das SIG-Zertifikat beim HTTP-Upgrade über H2-Cert übertragen, geprüft und zusammen mit dem TLS-Client-Zertifikat, der MP-ID, der Marktrolle und der `connectionId` an die Verbindung gebunden werden.

MUST Der Header-Parameter `kid` MUSS den Base64url-kodierten SHA-256-Hash des DER-kodierten SIG-Zertifikats enthalten und entspricht damit dem Wert des Parameters `x5t#S256` gemäß [RFC7515].

MUST NOT Das ENC-Zertifikat des EMT .API-Zertifikatstripels DARF in diesem Profil NICHT verwendet werden; eine zusätzliche Verschlüsselung fachlicher Nutzdaten auf Anwendungsebene erfolgt nicht.

3.3 Hashverfahren

MUST Das Hashverfahren des Signing Inputs ergibt sich aus dem Algorithmus (SHA-256 bei `urn:h2:jws:alg:brainpoolP256r1-sha256`, SHA-384 bei `urn:h2:jws:alg:brainpoolP384r1-sha384`). Für den im HTTP-Kontext verwendeten Header `H2-Digest` MUSS unabhängig davon stets SHA-256 verwendet werden (Kapitel 5.3).

4. Gemeinsame Signaturmechanik

Die Kapitel 4.1 bis 4.3 gelten sowohl für den HTTP-Kontext (Kapitel 5) als auch für den WebSocket-Kontext (Kapitel 6). Die Bildung der jeweiligen Signaturrepräsentation ist kontextspezifisch und wird in Kapitel 5.1 beziehungsweise 6.1 festgelegt.

4.1 Serialisierung

MUST Die JWS MUSS in Compact Serialization mit Detached Payload gebildet werden ([RFC7515], Anhang F). Der Payload MUSS unkodiert in den Signing Input einbezogen werden (`b64 = false` gemäß [RFC7797]); der Parameter `b64` MUSS im Parameter `crit` aufgeführt sein.

Die übertragene JWS besteht damit aus Protected Header und Signatur mit leerem Payload-Teil:

```
BASE64URL(UTF8(Protected Header)) || '.' || '' || '.' || BASE64URL(Signatur)
```

MUST Der Prüfer MUSS die Struktur der JWS validieren: genau zwei Punkte, ein nicht leerer erster und dritter Teil sowie ein leerer zweiter Teil. Abweichende Strukturen MÜSSEN abgelehnt werden.

4.2 JOSE-Header

Der Protected Header MUSS genau die folgenden Parameter enthalten:

Parameter	Pflicht	Beschreibung
<code>alg</code>	MUSS	<code>urn:h2:jws:alg:brainpoolP256r1-sha256</code> oder <code>urn:h2:jws:alg:brainpoolP384r1-sha384</code> (Kapitel 3.1).
<code>typ</code>	MUSS	<code>h2-http-sig+jws</code> im HTTP-Kontext (Kapitel 5), <code>h2-ws-sig+jws</code> im WebSocket-Kontext (Kapitel 6).
<code>b64</code>	MUSS	Fester Wert <code>false</code> gemäß [RFC7797].
<code>crit</code>	MUSS	Fester Wert <code>["b64"]</code> . Weitere kritische Header-Parameter DÜRFEN NICHT verwendet werden.
<code>kid</code>	MUSS	<code>x5t#S256</code> -Wert des Signaturzertifikats (Kapitel 3.2).

MUST NOT Andere als die in der Tabelle aufgeführten Parameter DÜRFEN im Protected Header NICHT enthalten sein; insbesondere DARF `x5c` NICHT verwendet werden. Der Protected Header ist ein geschlossenes Objekt; Nachrichten mit zusätzlichen Parametern MÜSSEN abgelehnt werden.

Der Protected Header trägt bewusst keine kontextbindenden Parameter (Transaktionskennung, Zeitpunkt, Methode, Ziel). Diese Bindung erfolgt ausschließlich über den Inhalt der jeweiligen Signaturrepräsentation gemäß Kapitel 5.1 beziehungsweise 6.1, die vollständig Teil des signierten Signing Input ist.

4.3 Kanonisierung und Signing Input

MUST Die Signaturrepräsentation MUSS I-JSON-konform gemäß [RFC7493] und UTF-8-kodiert ohne BOM sein (Teil 1, Kapitel 4.3 [API-GL]) und MUSS vor der Signaturbildung gemäß JCS [RFC8785] kanonisiert werden.

MUST NOT Der Protected Header, die Signaturrepräsentation und ein übertragener JSON-Payload DÜRFEN auf keiner Objektebene mehrfach vorkommende Eigenschaftsnamen enthalten. Der Prüfer MUSS Duplikate vor der JCS-Kanonisierung erkennen und die Nachricht ablehnen.

MUST NOT Ein JSON-Parser DARF bei mehrfachen Eigenschaftsnamen NICHT stillschweigend den ersten oder letzten Wert übernehmen und DARF vor der Duplikatprüfung keine semantische Normalisierung des JSON-Objekts durchführen.

MUST Der Signing Input MUSS wie folgt gebildet werden: ASCII(BASE64URL(UTF8(Protected Header))) gefolgt von einem Punkt (0x2E) gefolgt von JCS(Signaturrepräsentation).

MUST Die Signaturprüfung MUSS über die JCS-Kanonisierung der tatsächlich empfangenen beziehungsweise aus dem Nachrichtenkontext gebildeten Signaturrepräsentation erfolgen. Die Reihenfolge von JSON-Feldern und nicht signifikanter Whitespace im übertragenen Payload sind für die Gültigkeit der Signatur unerheblich (Teil 1, Kapitel 4.3 und 4.4.2 [API-GL]).

5. Signatur im HTTP-Kontext

Für diesen Kontext gelten die Vorgaben aus Teil 2, Kapitel 2.3 [API-GL]. Dieses Kapitel legt die dort geforderte konkrete JWS-Serialisierung, die Bildung der Signaturrepräsentation, die Normalisierung von URI und Query-Parametern, die Übertragung der Signatur sowie die Referenzierung des Signaturzertifikats fest.

5.1 Bestandteile der Signaturrepräsentation

Gemäß Teil 2, Kapitel 2.3 [API-GL] MÜSSEN Payload und Steuerinformationen zur Signaturbildung in einem JSON-Objekt aus den Elementen `meta` und `data` zusammengeführt werden. Dieses Objekt dient ausschließlich der Signaturerzeugung und -prüfung und wird nicht übertragen.

MUST Bei einem HTTP-Request MUSS das Element `meta` die normalisierte Request-URI einschließlich aller Query-Parameter, die HTTP-Methode, die aktuelle `H2-Transaction-Id` sowie die im Nachrichtenkontext vorhandenen signaturrelevanten HTTP-Header gemäß [API-GL] enthalten. Ein Feld `created` DARF NICHT enthalten sein.

MUST Eine asynchrone fachliche Response ist auf der jeweiligen Kommunikationsstrecke als neuer HTTP-Request in Gegenrichtung zu behandeln und MUSS das Request-Signaturprofil dieses Kapitels verwenden. Zusätzlich MUSS `H2-Reference-Id` die effektive Transaktions-ID des ursprünglichen Aufrufs auf derselben Kommunikationsstrecke enthalten.

MUST Bei einer synchronen HTTP-Response mit JSON-Body MUSS `meta` mindestens die normalisierte URI und die HTTP-Methode des konkret beantworteten Requests, den HTTP-Statuscode der Response als Ganzzahl, die eigene `H2-Transaction-Id` der Response, die `H2-Reference-Id`, `H2-API-Version`, `Content-Type` sowie, sofern ein ETag übertragen wird, dessen Wert enthalten. Ein Feld `created` DARF NICHT enthalten sein.

MUST Die `H2-Transaction-Id` der Response MUSS als UUID Version 7 neu erzeugt werden. Die `H2-Reference-Id` MUSS die effektive Transaktions-ID des konkret beantworteten Requests enthalten: bei einem erstmaligen Request dessen `H2-Transaction-Id`, bei einem Retry dessen `H2-Initial-Transaction-Id`.

MUST Der Prüfer MUSS URI, HTTP-Methode und die effektive Transaktions-ID des beantworteten Requests aus seinem lokal vorhandenen Request-Kontext bilden und mit den signierten Werten vergleichen.

MUST Der signierende API-Anbieter MUSS zum Zeitpunkt der Response-Erzeugung über den vollständigen Request-Kontext (normalisierte URI, HTTP-Methode, effektive Transaktions-ID) verfügen. Wird ein Request asynchron, über eine Zwischenkomponente oder in einem anderen Prozess als dem empfangenden verarbeitet, MUSS dieser Kontext unverändert und vertrauenswürdig bis zur Signaturerstellung der Response durchgereicht werden. Kann der Kontext nicht vertrauenswürdig hergestellt werden, MUSS die Verarbeitung fehlschlagen und DARF keine signierte Response erzeugt werden.

MUST Der technische Erstellungszeitpunkt der HTTP-Nachricht MUSS ausschließlich aus dem UUID-Version-7-Zeitwert ihrer aktuellen H2-Transaction-Id abgeleitet werden. Bei Retries ist die aktuelle H2-Transaction-Id und nicht H2-Initial-Transaction-Id maßgeblich; bei Responses ist die eigene H2-Transaction-Id und nicht H2-Reference-Id maßgeblich.

MUST Das Element data MUSS dem tatsächlich übertragenen Payload entsprechen. Wird ein leeres JSON-Objekt als Payload übertragen, MUSS data = {} gesetzt werden. Enthält der Request keinen Payload, MUSS das Element data ersatzlos entfallen; dies DARF NICHT mit data = {} gleichgesetzt werden (Teil 2, Kapitel 2.3 [API-GL]).

Beispiel der Signaturrepräsentation eines Requests mit Payload (nicht übertragen, dient ausschließlich der Signaturbildung):

```
{
  "meta": {
    "uri": "/9871000654321/H2API00001/1",
    "method": "POST",
    "H2-Transaction-Id": "019f5a0f-9b00-7321-8123-456789abcdef",
    "H2-Message-Sender": "9871000123456",
    "H2-Message-Sender-Role": "GridOperator",
    "H2-Message-Receiver": "9871000654321",
    "H2-Message-Receiver-Role": "DataHub",
    "H2-Business-Process": "preliminaryMeasurementSubmission",
    "Content-Type": "application/json"
  },
  "data": {
    "quantity": "1250.5",
    "periodStart": "2026-07-13T06:00:00Z"
  }
}
```

Beispiel der Signaturrepräsentation einer synchronen Response mit Payload (nicht übertragen, dient ausschließlich der Signaturbildung):

```
{
  "meta": {
    "uri": "/9871000654321/H2API00001/1",
    "method": "POST",
    "statusCode": 200,
    "H2-Transaction-Id": "019f5a0f-9ee8-7654-8234-56789abcdef0",
    "H2-Reference-Id": "019f5a0f-9b00-7321-8123-456789abcdef",
    "H2-API-Version": "1.0.0",
    "Content-Type": "application/json"
  },
  "data": {
    "status": "accepted"
  }
}
```

```
}  
}
```

5.2 Normalisierung der Request-URI

MUST Ersteller und Prüfer MÜSSEN die URI aus dem unveränderten HTTP-Request-Target in Origin-Form bilden: Pfad und, sofern vorhanden und nicht leer, ein Fragezeichen mit dem Query-String. Schema, Authority, Host und Fragment sind NICHT Bestandteil dieses Werts.

MUST Für Pfad und Query-String MÜSSEN gültige Prozent-Kodierungen auf Großbuchstaben normalisiert und Prozent-Kodierungen unreservierter Zeichen dekodiert werden. Ungültige oder unvollständige Prozent-Kodierungen MÜSSEN abgelehnt werden.

MUST Punkt-Segmente im Pfad MÜSSEN gemäß `remove_dot_segments` aus [RFC3986] entfernt werden. Ein leerer Pfad MUSS als `/` dargestellt werden.

MUST NOT Im Pfad DÜRFEN weder ein umgekehrter Schrägstrich noch ein Prozent-kodierter Schrägstrich oder umgekehrter Schrägstrich enthalten sein. Die Sequenzen `%2F` und `%5C` sind unabhängig von der Groß-/Kleinschreibung abzulehnen und DÜRFEN NICHT dekodiert werden.

MUST Im Query-String MÜSSEN Reihenfolge, Wiederholungen und Schreibweise der Parameter nach der vorstehenden Prozent-Normalisierung erhalten bleiben. Parameter DÜRFEN NICHT sortiert, zusammengefasst oder dedupliziert werden.

MUST Ein Pluszeichen `+` MUSS als wörtliches Pluszeichen behandelt werden und DARF NICHT in ein Leerzeichen umgewandelt werden. Ein Leerzeichen MUSS als `%20` repräsentiert sein.

MUST Ein Parameter ohne Gleichheitszeichen und ein Parameter mit leerem Wert sind verschieden: `a` und `a=` MÜSSEN unverändert unterscheidbar bleiben. Leere Parameternamen, leere Query-Segmente sowie aufeinanderfolgende oder abschließende Trennzeichen `&` MÜSSEN abgelehnt werden.

MUST Ist die Query-Komponente nicht vorhanden oder leer, DARF die kanonische URI kein abschließendes Fragezeichen enthalten. Ein Fragment DARF weder übertragen noch in die Signaturrepräsentation aufgenommen werden.

MUST Wiederholte Query-Parameter sind nur zulässig, wenn die jeweilige API-Spezifikation sie zulässt. Sind sie zulässig, MUSS ihre ursprüngliche Reihenfolge Bestandteil der Signatur bleiben.

MUST Die Signaturprüfung MUSS am TLS-Terminierungspunkt oder anhand dort unveränderlich erfasster Rohdaten vor jeder Pfaddekodierung, Routenauflösung oder Umschreibung erfolgen. Werte aus `Forwarded`, `X-Forwarded-*` oder aus rekonstruierten Framework-Routen DÜRFEN NICHT zur Bildung der signierten URI verwendet werden.

MUST Verändert ein Gateway oder Proxy Pfad oder Query-String für eine nachfolgende Kommunikationsstrecke, MUSS es die eingehende Signatur vor der Veränderung prüfen und für die ausgehende Strecke eine neue Signatur über den dort tatsächlich verwendeten Request-Target erzeugen.

5.2.1 Kanonisierung signaturrelevanter HTTP-Header

MUST Jeder signaturrelevante HTTP-Header MUSS im jeweiligen Nachrichtenkontext höchstens einmal als einzelnes Feld vorkommen. Mehrere Feldzeilen, durch Kommata zusammengeführte Mehrfachwerte oder voneinander abweichende Groß-/Kleinschreibungen desselben Feldnamens MÜSSEN abgelehnt werden.

MUST HTTP-Feldnamen sind auf dem Übertragungsweg unabhängig von Groß-/Kleinschreibung. In meta MÜSSEN sie ausschließlich in der in [API-GL] und dieser Spezifikation festgelegten kanonischen Schreibweise verwendet werden.

MUST NOT Signaturrelevante Headerwerte DÜRFEN keine führenden oder nachfolgenden optionalen Leerzeichen, keine Zeilenfaltung und keine Steuerzeichen enthalten. Solche Nachrichten MÜSSEN abgelehnt und DÜRFEN NICHT durch Trimmen oder Entfalten repariert werden.

MUST Der Header Content-Type MUSS syntaktisch als `application/json` mit optionalem Parameter `charset=utf-8` validiert werden. In der Signaturrepräsentation MUSS unabhängig von optionalen Leerzeichen und Groß-/Kleinschreibung der Token stets der kanonische Wert `application/json` verwendet werden.

MUST Die Werte von If-Match, If-None-Match und ETag MÜSSEN nach erfolgreicher Syntaxprüfung exakt in ihrer kanonischen starken Form `"rev-<revision>"` beziehungsweise als fester Wert `*` in die Signaturrepräsentation aufgenommen werden.

MUST Ein signaturrelevanter Header, der im Nachrichtenkontext nicht vorhanden ist, MUSS in meta vollständig entfallen. Ein fehlender Header, ein leerer Headerwert und ein Headerwert mit Leerzeichen DÜRFEN NICHT gleichgesetzt werden.

5.3 Übertragung in HTTP-Headern

Die Signatur wird gemäß der in Teil 1, Kapitel 4.5.1 [API-GL] festgelegten Namenskonvention (Präfix H2-, kein X-Präfix) in den folgenden Headern übertragen. Dies entspricht funktional den BDEW-Headern X-BDEW-SIGNATURE, X-BDEW-DIGEST und X-BDEW-CERT [RzÜ-API]:

HTTP-Header	Pflicht	Beschreibung
H2-Signature	MUSS	Die JWS in Compact Serialization mit Detached Payload (Kapitel 4.1). Maximale Länge: 8.192 Byte.
H2-Digest	MUSS	Base64-kodierter (Kapitel 4 [RFC4648]) SHA-256-Hash über JCS(data). Ist data nicht vorhanden, MUSS der Hash des leeren Oktettstroms verwendet werden (47DEQpj8HBSa+/TImW+5JCeuQeRkm5NMpJWZG3hSuFU=). Maximale Länge: exakt 44 Zeichen (Base64 eines SHA-256-Werts).

HTTP-Header	Pflicht	Beschreibung
H2-Cert	MUSS	Das DER-kodierte Signaturzertifikat als Byte Sequence Structured Field gemäß Kapitel 2.1 [RFC9440] (Base64 zwischen Doppelpunkten). Maximale Länge: 16.384 Byte. Im HTTP-Upgrade-Request einer WebSocket-Verbindung wird H2-Cert ohne H2-Signature und H2-Digest übertragen und dient dort ausschließlich der Bindung des SIG-Zertifikats an die Verbindung ([WS-SPEC], Kapitel 3.4); die Längenbegrenzung gilt unverändert.

MUST H2-Digest MUSS ausschließlich über das Element data gebildet werden und dient der frühzeitigen Integritätsprüfung des Payloads unabhängig von der vollständigen Signaturverifikation. Die maßgebliche Signatur MUSS sich stets auf die vollständige Signaturrepräsentation aus meta und data beziehen.

MUST Fachlich relevante und damit signaturpflichtige synchrone HTTP-Responses mit JSON-Body MÜSSEN die Header H2-Transaction-Id, H2-Reference-Id, H2-API-Version, H2-Signature, H2-Digest, H2-Cert und Content-Type tragen. Ersteller ist der API-Anbieter der aktuellen Kommunikationsstrecke.

MUST Die Elemente method und uri in meta MÜSSEN Methode und normalisierte Request-URI des beantworteten Requests enthalten. statusCode MUSS den tatsächlichen HTTP-Statuscode der Response enthalten.

MUST NOT Synchrone HTTP-Responses ohne Body sowie sämtliche HTTP-Responses des WebSocket-Upgrade-Verfahrens DÜRFEN die Header H2-Signature, H2-Digest und H2-Cert NICHT enthalten.

6. Signatur im WebSocket-Kontext

Für diesen Kontext gelten gemäß Teil 2, Kapitel 5.2 [API-GL] die Vorgaben aus Kapitel 2.3 [API-GL] entsprechend. Da eine WebSocket-Nachricht keine HTTP-Methode, keine Request-URI und keine HTTP-Header je Nachricht besitzt, legt dieses Kapitel die entsprechende Anwendung verbindlich fest.

6.1 Der Envelope als Signaturrepräsentation

MUST Für signaturpflichtige Nachrichten gemäß [WS-SPEC], Kapitel 3.5, MUSS die Signaturrepräsentation ein geschlossenes JSON-Objekt aus den Eigenschaften `connectionId` und `message` sein. `connectionId` MUSS der H2-Transaction-Id des erfolgreichen HTTP-Upgrade-Requests der aktuellen Verbindung entsprechen. `message` MUSS den vollständigen Nachrichtenrahmen (Envelope) OHNE die Eigenschaft `signature` enthalten.

Die folgende Tabelle stellt die entsprechende Anwendung der Bestandteile aus Kapitel 5.1 auf die WebSocket-Signaturrepräsentation dar:

Bestandteil gemäß Kapitel 2.3 [API-GL] / 5.1	Entsprechung im WebSocket-Envelope
Bindung an den Verbindungsaufbau	<code>connectionId</code> = H2-Transaction-Id des erfolgreichen Upgrade-Requests ([WS-SPEC], Kapitel 3.3 und 3.5)
Adressierte Ressource / fachliche Operation	<code>type</code> (Nachrichtentyp, [WS-SPEC] Kapitel 4.3)
Eindeutige Transaktionskennung	<code>message.messageId</code> ([WS-SPEC], Kapitel 4.2); innerhalb der Verbindung eindeutig und nach erfolgreicher Signaturprüfung atomar zu reservieren
Technischer Erstellungszeitpunkt	<code>message.timestamp</code> gemäß [WS-SPEC]; im HTTP-Kontext wird der Zeitpunkt aus der aktuellen H2-Transaction-Id abgeleitet.
Payload (data)	<code>message.payload</code> ([WS-SPEC], Kapitel 4.2)
H2-Message-Sender, H2-Message-Sender-Rolle u. a.	entfällt; die Absenderidentität ergibt sich aus der mTLS-Authentisierung der Verbindung (Kapitel 6.2)
HTTP-Methode, Request-URI, HTTP-Statuscode	entfällt; ohne technische Entsprechung innerhalb einer bestehenden WebSocket-Verbindung

MUST `correlationId` MUSS als Bestandteil von `message` in der Signaturrepräsentation enthalten sein, obwohl sie bei signaturpflichtigen Client-Nachrichten stets den Wert `null` führt ([WS-SPEC], Kapitel 11.3).

Beispiel der Signaturrepräsentation einer subscribe-Nachricht; das Objekt wird ausschließlich zur Signaturbildung erzeugt und nicht übertragen:

```
{
  "connectionId": "019f55a9-81c0-7456-8345-6789abcdef01",
  "message": {
    "type": "subscribe",
    "messageId": "3f2a6c1e-9d4b-4c1a-8e2f-5b7d9a0c3e11",
    "correlationId": null,
    "timestamp": "2026-07-12T09:30:00Z",
    "payload": {
      "providerId": "9910123400002",
      "apiId": "H2API00001",
      "majorVersion": 1
    }
  }
}
```

MUST Der übertragene Envelope bleibt unverändert. `connectionId` wird nicht als Envelope-Eigenschaft übertragen, sondern aus dem Verbindungsaufbau bezogen und bei der Signaturerstellung beziehungsweise -prüfung ergänzt.

Die `connectionId` ist die H2-Transaction-Id des erfolgreich abgeschlossenen Upgrade-Requests gemäß [WS-SPEC], Kapitel 3.3; eine eigenständige Kennung wird nicht erzeugt.

6.2 Signaturzertifikat der WebSocket-Verbindung

MUST Der Client MUSS im HTTP-Upgrade-Request den Header `H2-Cert` mit dem DER-kodierten SIG-Zertifikat seines `EMT.API`-Zertifikatstripels übertragen. Der Server MUSS Zertifikatsklasse, Kette, Gültigkeit, Sperrstatus, Verwendungszweck und MP-ID prüfen und das Zertifikat an den Verbindungskontext binden.

MUST Der Header-Parameter `kid` jeder WebSocket-JWS MUSS den `x5t#S256`-Wert des beim Upgrade gebundenen SIG-Zertifikats enthalten. Der Prüfer MUSS `kid` gegen dieses Zertifikat abgleichen.

Die Zuordnung des SIG-Zertifikats und des TLS-Client-Zertifikats zum selben Zertifikatstripel und zur selben MP-ID sowie die Einbeziehung der `connectionId` binden Signatur, Nachricht und konkreten Kommunikationskanal aneinander. Eine Signatur mit einem anderen Zertifikat oder für eine andere Verbindung ist ungültig.

MUST Die MP-ID im OU-Feld des SIG-Zertifikats MUSS mit der MP-ID im TLS-Client-Zertifikat und mit der beim Verbindungsaufbau festgestellten MP-ID übereinstimmen. Ein Wechsel des SIG- oder TLS-Zertifikats erfordert einen neuen Verbindungsaufbau.

6.3 Übertragung der Signatur

MUST Die Signatur MUSS in der Envelope-Eigenschaft `signature` der signaturpflichtigen Nachricht übertragen werden ([WS-SPEC], Kapitel 4.2 und 11.3). Die Eigenschaft `signature` selbst ist NICHT Bestandteil von `message` in der Signaturrepräsentation gemäß Kapitel 6.1.

MUST NOT Ein gesonderter Digest (entsprechend `H2-Digest` im HTTP-Kontext) wird im WebSocket-Kontext NICHT gebildet und NICHT übertragen; die Signatur selbst deckt den vollständigen Envelope ab.

7. Signaturerstellung und Signaturprüfung

7.1 Signaturerstellung

Der Ersteller MUSS die Signatur in folgenden Schritten bilden:

1. Signaturrepräsentation bilden: im HTTP-Kontext das Objekt aus meta und data gemäß Kapitel 5.1, im WebSocket-Kontext das Objekt aus connectionId und message gemäß Kapitel 6.1.
2. Signaturrepräsentation als I-JSON erzeugen und gemäß JCS [RFC8785] kanonisieren.
3. Protected Header gemäß Kapitel 4.2 erstellen (alg, typ, b64, crit, kid).
4. Signing Input gemäß Kapitel 4.3 bilden.
5. Signatur mit dem privaten Schlüssel des Signaturzertifikats gemäß alg berechnen und als R || S Base64url-kodieren.
6. JWS in Compact Serialization mit leerem Payload-Teil zusammensetzen.
7. Im HTTP-Kontext: JWS in H2-Signature übertragen, H2-Digest über JCS(data) und H2-Cert gemäß Kapitel 5.3 setzen; den unveränderten Payload als HTTP-Body übertragen. Im WebSocket-Kontext: JWS in der Envelope-Eigenschaft signature gemäß Kapitel 6.3 übertragen.

7.2 Signaturprüfung

Der Prüfer MUSS eine empfangene Nachricht in folgenden Schritten verifizieren. Schlägt ein Schritt fehl, MUSS die Prüfung abgebrochen und die Nachricht gemäß Kapitel 9 abgelehnt werden; eine fachliche Verarbeitung DARF NICHT erfolgen.

1. Vorhandensein und syntaktische Gültigkeit der Signatur prüfen (H2-Signature/H2-Digest/H2-Cert im HTTP-Kontext; signature im WebSocket-Kontext) sowie die Längenbegrenzungen gemäß Kapitel 5.3 beziehungsweise Kapitel 11 bzw. [WS-SPEC] Kapitel 10.
2. Struktur der JWS gemäß Kapitel 4.1 prüfen und den Protected Header dekodieren; alg, typ, b64 und crit gegen Kapitel 4.2 prüfen.
3. Signaturschlüssel referenzieren: im HTTP-Kontext das SIG-Zertifikat aus H2-Cert dekodieren und prüfen; im WebSocket-Kontext das beim Upgrade gemäß Kapitel 6.2 gebundene SIG-Zertifikat verwenden. In beiden Kontexten sind Gültigkeit, Zertifikatsklasse, Zertifikatstripel-Zuordnung, Kette zur SM-PKI, Verwendungszweck und Sperrstatus zu prüfen.
4. kid mit dem x5t#S256-Wert des referenzierten Zertifikats vergleichen.
5. Identität des technischen Signierers prüfen:

- a) HTTP-Request ohne H2-Forwarded-By: Die MP-ID im OU-Feld des Signaturzertifikats MUSS mit H2-Message-Sender übereinstimmen.
 - b) HTTP-Request mit H2-Forwarded-By: Die MP-ID im OU-Feld des Signaturzertifikats MUSS mit H2-Forwarded-By übereinstimmen. H2-Forwarded-By MUSS der für die aktuelle Umgebung konfigurierten MP-ID des Data Hub entsprechen. H2-Message-Sender und H2-Message-Sender-Role bezeichnen weiterhin den ursprünglichen fachlichen Absender und sind durch die Signatur gegen Veränderung geschützt.
 - c) Synchrone HTTP-Response: Die MP-ID im OU-Feld des Signaturzertifikats MUSS mit der MP-ID des über die aktuelle mTLS-Verbindung authentisierten API-Anbieters übereinstimmen.
 - c.1) Bei einer synchronen HTTP-Response MUSS der Prüfer uri, method und H2-Reference-Id mit dem lokal vorliegenden Request-Kontext vergleichen. H2-Reference-Id MUSS der effektiven Transaktions-ID des beantworteten Requests entsprechen.
 - c.2) Die H2-Transaction-Id der Response MUSS eine syntaktisch gültige UUID Version 7 sein. Der Replay-Schutz gilt als erfüllt, wenn der Client die Response genau einem eigenen offenen Request zuordnet und diesen Request nach Annahme genau einer gültigen Response abschließt; ein gesonderter Replay-Speicher gemäß Kapitel 8.2 ist für synchrone Responses nicht erforderlich.
 - d) WebSocket-Nachricht: Zertifikat und MP-ID sind gemäß Kapitel 6.2 bereits an den Sicherheitskontext der Verbindung gebunden.
6. Empfangene beziehungsweise aus dem Nachrichtenkontext gebildete Signaturrepräsentation gemäß JCS kanonisieren, den Signing Input gemäß Kapitel 4.3 bilden und die Signatur mit dem öffentlichen Schlüssel verifizieren.
 7. Im HTTP-Kontext zusätzlich: H2-Digest mit dem selbst berechneten SHA-256-Hash über JCS(data) vergleichen.
 8. Zeitfenster prüfen: im HTTP-Kontext den Zeitwert aus der aktuellen H2-Transaction-Id gemäß UUID Version 7, im WebSocket-Kontext den signierten timestamp gegen Kapitel 8.1 prüfen.
 9. Replay- und Einmaligkeitsprüfung gemäß Kapitel 8.2 durchführen. Die dauerhafte Reservierung eines Replay-Schlüssels oder einer messageId DARF erst nach erfolgreicher Signatur- und Zeitfensterprüfung erfolgen.

8. Replay-Schutz und Zeitfenster

8.1 Zeitfenster

MUST Der maßgebliche technische Erstellungszeitpunkt DARF beim Beginn der Prüfung nicht älter als 300 Sekunden und nicht mehr als 60 Sekunden in der Zukunft liegen. Nachrichten außerhalb dieser Grenzen MÜSSEN abgelehnt werden.

Das Zeitfenster wird für jede Nachricht einzeln anhand ihrer eigenen aktuellen H2-Transaction-Id geprüft. Request und zugehörige Response teilen kein gemeinsames Zeitfenster; eine Response mit eigener, frisch erzeugter H2-Transaction-Id besteht die Prüfung auch dann, wenn die Verarbeitung des Requests länger als 300 Sekunden gedauert hat. Das Signatur-Zeitfenster begründet keine fachliche Gesamt-Timeout-Semantik für einen Vorgang.

MUST Alle Systeme, die signaturpflichtige Nachrichten erzeugen oder prüfen, MÜSSEN ihre Systemzeit über eine zuverlässige Zeitquelle (z. B. NTP) synchronisieren. Die Uhrenabweichung MUSS deutlich unterhalb des maximalen Zukunftsversatzes von 60 Sekunden liegen.

Im HTTP-Kontext wird der Zeitpunkt aus dem 48-Bit-Unix-Zeitwert der aktuellen H2-Transaction-Id abgeleitet. Bei einem Retry ist die aktuelle H2-Transaction-Id und nicht H2-Initial-Transaction-Id maßgeblich; bei einer Response ist ihre eigene H2-Transaction-Id und nicht H2-Reference-Id maßgeblich. Im WebSocket-Kontext ist der signierte timestamp maßgeblich.

8.2 Einmaligkeit

MUST Im HTTP-Kontext MUSS der Prüfer die Wiederverwendung einer H2-Transaction-Id innerhalb des zulässigen Signaturzeitfensters erkennen.

Dieses Kapitel gilt für HTTP-Requests, Retry-Requests und asynchrone fachliche Responses. Für synchrone HTTP-Responses gilt die vereinfachte Replay-Prüfung über die Zuordnung zum eigenen offenen Request gemäß Kapitel 7.2.

MUST Der Replay-Schlüssel MUSS mindestens aus Umgebung, MP-ID des technischen Signierers und H2-Transaction-Id bestehen. Prüfung und Reservierung MÜSSEN atomar und bei horizontal skaliertem Verarbeitung clusterweit wirksam sein.

MUST Die Reservierung DARF erst nach erfolgreicher Prüfung von Zertifikat, Identität, Signatur, Digest und Zeitfenster erfolgen.

MUST Eine erneut empfangene Nachricht mit derselben H2-Transaction-Id DARF den fachlichen Vorgang NICHT erneut ausführen. Stimmen HTTP-Methode, normalisierte URI, signaturrelevante Header und Payload-Hash mit der zuerst empfangenen Nachricht überein, MUSS das Verhalten den Idempotenzregeln aus Teil 1, Kapitel 4.5.3 [API-GL] entsprechen. Weicht mindestens eines dieser Merkmale ab, MUSS die Nachricht mit 409 Conflict abgelehnt werden.

MUST Der Replay-Schlüssel MUSS mindestens bis zu dem spätesten Zeitpunkt gespeichert werden, zu dem dieselbe signierte Nachricht noch die Zeitfensterprüfung bestehen könnte. Bei maximal 300 Sekunden Alter und 60 Sekunden Zukunftsversatz beträgt die Mindestaufbewahrung 360 Sekunden ab der erstmaligen Annahme.

MUST Kann die atomare Replay-Prüfung nicht durchgeführt werden, MUSS der Prüfer die Nachricht ohne fachliche Verarbeitung mit 503 Service Unavailable ablehnen.

MUST Der für die atomare Replay-Prüfung eingesetzte Speicher MUSS hochverfügbar und bei horizontal skaliertem Verarbeitung clusterweit konsistent betrieben werden. Seine Nichtverfügbarkeit führt gemäß diesem Kapitel zur Ablehnung sämtlicher signaturpflichtiger Nachrichten mit 503 Service Unavailable und MUSS betrieblich überwacht und alarmiert werden.

MUST Im WebSocket-Kontext MUSS die Signaturrepräsentation die `connectionId` der aktuellen Verbindung enthalten. Eine auf einer anderen Verbindung aufgezeichnete Nachricht MUSS deshalb bereits bei der Signaturprüfung ungültig sein.

MUST Innerhalb derselben WebSocket-Verbindung MUSS die `messageId` eindeutig sein. Der Server MUSS die `messageId` nach erfolgreicher Signaturprüfung atomar reservieren und eine Wiederverwendung gemäß [WS-SPEC], Kapitel 4.2, mit ERR-MSG-002 ablehnen.

MUST NOT Eine Nachricht mit fehlender oder ungültiger Signatur DARF eine `messageId` NICHT dauerhaft reservieren.

MUST NOT Eine gültige Signatur DARF NICHT für eine andere als die signierte Repräsentation akzeptiert werden. Im HTTP-Kontext ist damit die Wiederverwendung einer Signatur für einen anderen Endpunkt, eine andere Methode oder einen anderen Statuscode ausgeschlossen; im WebSocket-Kontext die Wiederverwendung für eine andere Verbindung, einen anderen Nachrichtentyp oder eine andere `messageId`.

Übersicht der Einmaligkeits- und Zuordnungsmechanismen (informativ):

- HTTP-Replay-Schutz (Kapitel 8.2): Schlüssel Umgebung + MP-ID des technischen Signierers + H2-Transaction-Id; gilt für Requests, Retry-Requests und asynchrone fachliche Responses; Aufbewahrung mindestens 360 Sekunden; Verstoß führt zur Ablehnung der Nachricht.
- Synchrone HTTP-Responses (Kapitel 7.2): Zuordnung zum eigenen offenen Request über `uri`, `method` und H2-Reference-Id; kein gesonderter Replay-Speicher; nicht zuordenbare oder mehrfach eintreffende Responses werden verworfen.
- Fachliche Idempotenz (Teil 1, Kapitel 4.5.3 [API-GL]): Schlüssel H2-Message-Sender + effektive Transaktions-ID; verhindert die erneute fachliche Ausführung bei Retries; Aufbewahrung mindestens Retry-Zeitraum zuzüglich maximaler Verarbeitungsdauer.
- WebSocket-`messageId` ([WS-SPEC], Kapitel 4.2): Einmaligkeit je Verbindung; Vorhaltung für die gesamte Verbindungsdauer; Verstoß führt zu ERR-MSG-002.

Legitime Wiederholungen einer fachlichen HTTP-Operation (Retry-Requests gemäß Teil 1, Kapitel 4.5.3 [API-GL]) erhalten eine neue H2-Transaction-Id und damit eine neue Signatur; die Idempotenz wird über H2-Initial-Transaction-Id hergestellt.

9. Fehlerbehandlung

9.1 HTTP-Kontext

Fehler der Signaturprüfung werden mit den in diesem Kapitel festgelegten HTTP-Statuscodes beantwortet. Diese Fehlerresponses sind technische Responses und werden NICHT auf Inhaltsebene signiert. Die Header H2-Signature, H2-Digest und H2-Cert DÜRFEN NICHT enthalten sein.

Negative Responses mit Response-Body MÜSSEN dem ErrorResponse-Schema gemäß Teil 1, Kapitel 4.4.3 [API-GL] entsprechen und eine neu erzeugte H2-Transaction-Id der Response tragen. transactionId im ErrorResponse-Body und H2-Reference-Id MÜSSEN denselben Wert enthalten. Grundsätzlich ist dies die effektive Transaktions-ID des abgelehnten Requests. Kann diese wegen der fehlenden oder fehlgeschlagenen Signaturprüfung nicht vertrauenswürdig bestimmt werden, MÜSSEN transactionId und H2-Reference-Id die syntaktisch gültige aktuelle H2-Transaction-Id des abgelehnten Übertragungsversuchs enthalten. Liegt keine syntaktisch gültige H2-Transaction-Id vor, MUSS die Ablehnung ohne JSON-Body und ohne H2-Reference-Id erfolgen.

Fehlerfall	HTTP-Statuscode
Fehlender, zu langer oder syntaktisch ungültiger Header H2-Signature, H2-Digest oder H2-Cert; ungültige JWS-Struktur; unzulässiger oder unbekannter Wert in alg, typ, b64 oder crit	400
Signatur kryptographisch ungültig; H2-Digest-Abweichung; Zertifikat ungültig, gesperrt, falsche Klasse oder kid-Abweichung; Verletzung des Zeitfensters (Kapitel 8.1)	401
MP-ID-Abweichung zwischen Signaturzertifikat und dem nach Kapitel 7.2 maßgeblichen technischen Signierer; unzulässiger oder nicht vertrauenswürdiger Wert in H2-Forwarded-By	403
Wiederverwendung einer H2-Transaction-Id mit abweichender Methode, URI, signaturrelevanten Headern oder abweichendem Payload-Hash	409
Die atomare oder clusterweite Replay-Prüfung steht nicht verlässlich zur Verfügung	503

9.2 WebSocket-Kontext

MUST Fehlende, strukturell ungültige oder kryptographisch ungültige Signaturen, eine abweichende connectionId sowie eine Verletzung des Zeitfensters gemäß Kapitel 8.1 MUSS der Server mit dem Fehlercode ERR-SIG-001 gemäß [WS-SPEC], Kapitel 3.5 und 8.2, beantworten. Der

Zustand von Subscriptions und Verfügbarkeitsmeldungen DARF hierdurch NICHT verändert werden.

MUST NOT Fehlerantworten DÜRFEN in keinem der beiden Kontexte Rückschlüsse auf kryptographische Interna zulassen, die über die Benennung des Fehlerfalls hinausgehen (z. B. keine Teilergebnisse der Verifikation).

MUST Schlägt die Signaturprüfung, die Bindung an den beantworteten Request oder die Replay-Prüfung einer synchronen HTTP-Response beim Client fehl, MUSS der Client die Response verwerfen und den Vorgang als fehlgeschlagen behandeln; eine fachliche Verwendung des Response-Payloads DARF NICHT erfolgen.

10. Beispiele

Die folgenden Beispiele sind nicht normativ.

10.1 Beispiel: HTTP-Request mit Payload

Signaturrepräsentation (meta/data, siehe Kapitel 5.1) in JCS-kanonisierter Form (eine Zeile, Schlüssel sortiert, ohne Whitespace):

```
{"data":{"periodStart":"2026-07-13T06:00:00Z","quantity":"1250.5"},"meta":{"Content-Type":"application/json","H2-Business-Process":"preliminaryMeasurementSubmission","H2-Message-Receiver":"9871000654321","H2-Message-Receiver-Role":"DataHub","H2-Message-Sender":"9871000123456","H2-Message-Sender-Role":"GridOperator","H2-Transaction-Id":"019f5a0f-9b00-7321-8123-456789abcdef","method":"POST","uri":"/9871000654321/H2API00001/1"}}
```

Resultierende HTTP-Header (Signatur- und Zertifikatswerte gekürzt):

```
H2-Signature: eyJhbGciOiJIc246aDI6andzOmFsZzpicmFpbmBvb2xQMjU2cjEtc2hhMjU2IiwidHlwIjoiaDItaHR0cC1zaWcrandzIiwiaWYjY0IjpmYWxzZSwiY3JpdCI6WyJiNjQiXSwia2lkIjoiblNswt-BuWjgtZXhhbXBsZSJ9..AQIDBAUGBwgJCgsMDQ4PEBESExQVFhcYGRobHB0eHyAhIiMkJSYnKCKqKywtLi8wMTIzNDU2Nzg5Ojs8PT4_QA (Beispielwert; Signaturbytes nicht aus einem realen Schlüssel erzeugt)
H2-Digest: a+bWUIFG1LML8Z7seUllIRMjrHSPzsMQT9k527iAoNg=
H2-Cert: :MIIC2jCCAcKgAwIBAgIB...gekuerzt...
```

10.2 Beispiel: signaturpflichtige WebSocket-Nachricht

Signaturrepräsentation aus connectionId und message gemäß Kapitel 6.1 in JCS-kanonisierter Form:

```
{"connectionId":"019f55a9-81c0-7456-8345-6789abcdef01","message":{"correlationId":null,"messageId":"3f2a6c1e-9d4b-4c1a-8e2f-5b7d9a0c3e11","payload":{"apiId":"H2API00001","majorVersion":1,"providerId":"9910123400002"},"timestamp":"2026-07-12T09:30:00Z","type":"subscribe"}}
```

Übertragene Nachricht; connectionId ist nicht Bestandteil des Envelopes, signature ist gekürzt:

```
{
  "type": "subscribe",
  "messageId": "3f2a6c1e-9d4b-4c1a-8e2f-5b7d9a0c3e11",
  "correlationId": null,
  "timestamp": "2026-07-12T09:30:00Z",
```

```
"payload": {  
  "providerId": "9910123400002",  
  "apiId": "H2API00001",  
  "majorVersion": 1  
},  
"signature":  
"eyJhbGciOiJI1cm46aDI6andzOmFsZzpicmFpbnBvb2xQMjU2cjEtc2hhMjU2IiwidHlwIjoiaDItd3Mtc2lnK2p3cyIsImI2NCI6ZmFsc2UsImNyaXQiOlsiYjY0Il0sImtpZCI6Im5TbFkwblo4LWV4YW1wbGUifQ..gekuerzt"
```

11. Betriebsparameter

Die folgenden Parameter gelten verbindlich für beide Kontexte, soweit nicht anders vermerkt. Änderungen erfolgen ausschließlich im Rahmen des Änderungsmanagements gemäß Teil 1, Kapitel 4.2 [API-GL].

Parameter	Wert	Beschreibung
Zulässige Algorithmen	urn:h2:jws:alg:brainpoolP256r1-sha256, urn:h2:jws:alg:brainpoolP384r1-sha384	JOSE-Algorithmen gemäß Kapitel 3.1; Vorgaben der [TR03116-3].
typ-Wert (HTTP)	h2-http-sig+jws	Fester Wert des Header-Parameters typ im HTTP-Kontext (Kapitel 4.2).
typ-Wert (WebSocket)	h2-ws-sig+jws	Fester Wert des Header-Parameters typ im WebSocket-Kontext (Kapitel 4.2).
Zeitfenster (Replay-Schutz)	max. 300 s alt / max. 60 s zukünftig	HTTP: UUIDv7-Zeitwert der aktuellen H2-Transaction-Id; WebSocket: timestamp (Kapitel 8.1).
HTTP-Replay-Schlüssel	Umgebung + MP-ID des technischen Signierers + H2-Transaction-Id	Clusterweit atomar zu prüfen und zu reservieren (Kapitel 8.2).
Mindestaufbewahrung HTTP-Replay	360 Sekunden ab Erstannahme	Mindestdauer bei 300 Sekunden Maximalalter und 60 Sekunden maximalem Zukunftsversatz (Kapitel 8.2). Gilt für Requests, Retry-Requests und asynchrone fachliche Responses, nicht für synchrone Responses (Kapitel 7.2).
WebSocket-Verbindungsbindung	connectionId = H2-Transaction-Id des Upgrade-Requests	Bestandteil der Signaturrepräsentation, aber nicht des übertragenen Envelopes (Kapitel 6.1).
Digest-Verfahren (H2-Digest)	SHA-256	Hash über JCS(data), Base64-kodiert (Kapitel 5.3, nur HTTP-Kontext).
Maximale Länge H2-Signature	8.192 Byte	Obergrenze des Header-Werts (nur HTTP-Kontext); größere Werte MÜSSEN mit 400 abgelehnt werden.
Maximale Länge H2-Cert	16.384 Byte	Obergrenze des Header-Werts (nur HTTP-Kontext); größere Werte MÜSSEN mit 400 abgelehnt werden.

Parameter	Wert	Beschreibung
Maximale Länge signature	4.096 Byte	Obergrenze der Envelope-Eigenschaft signature (nur WebSocket-Kontext, [WS-SPEC] Kapitel 10).
CRL-Aktualität	24 Stunden	Maximales Alter der bei der Prüfung verwendeten Sperrliste gemäß [CP-SM-PKI].

12. Quellen

- [API-GL] API-Guideline Wasserstoffmarkt, Version 1.0a, Working Document, in der jeweils gültigen Fassung; insbesondere Teil 1, Kapitel 4.2 bis 4.5, sowie Teil 2, Kapitel 2, 3 und 5.
- [WS-SPEC] Technische Schnittstellenbeschreibung – WebSocket-API des Verzeichnisdienstes, Version 1.0, Working Document, in der jeweils gültigen Fassung; insbesondere Kapitel 3.4, 3.5, 4.2 und 11.
- [RFC2119] Bradner, S. Key words for use in RFCs to Indicate Requirement Levels. IETF RFC 2119, March 1997. <https://www.rfc-editor.org/info/rfc2119>
- [RFC8174] Leiba, B. Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. IETF RFC 8174, May 2017. <https://www.rfc-editor.org/info/rfc8174>
- [RFC7515] Jones, M., Bradley, J., and Sakimura, N. JSON Web Signature (JWS). IETF RFC 7515, May 2015. <https://www.rfc-editor.org/info/rfc7515>
- [RFC7797] Jones, M. JSON Web Signature (JWS) Unencoded Payload Option. IETF RFC 7797, February 2016. <https://www.rfc-editor.org/info/rfc7797>
- [RFC8785] Rundgren, A., Jordan, B., and Erdtman, S. JSON Canonicalization Scheme (JCS). IETF RFC 8785, June 2020. <https://www.rfc-editor.org/info/rfc8785>
- [RFC7493] Bray, T. The I-JSON Message Format. IETF RFC 7493, March 2015. <https://www.rfc-editor.org/info/rfc7493>
- [RFC3339] Klyne, G. and Newman, C. Date and Time on the Internet: Timestamps. IETF RFC 3339, July 2002. <https://www.rfc-editor.org/info/rfc3339>
- [RFC3986] Berners-Lee, T., Fielding, R., and Masinter, L. Uniform Resource Identifier (URI): Generic Syntax. IETF RFC 3986, January 2005, insbesondere Kapitel 6.2.2 und 6.2.3. <https://www.rfc-editor.org/info/rfc3986>
- [RFC4648] Josefsson, S. The Base16, Base32, and Base64 Data Encodings. IETF RFC 4648, October 2006. <https://www.rfc-editor.org/info/rfc4648>
- [RFC9440] Campbell, B. and Bishop, M. Client-Cert HTTP Header Field. IETF RFC 9440, July 2023. <https://www.rfc-editor.org/info/rfc9440>
- [RFC9562] Davis, K., Peabody, B., and Leach, P. Universally Unique IDentifiers (UUIDs). IETF RFC 9562, May 2024. <https://www.rfc-editor.org/info/rfc9562>
- [RFC9864] Jones, M. and Steele, O. Fully-Specified Algorithms for JSON Object Signing and Encryption (JOSE) and CBOR Object Signing and Encryption (COSE). IETF RFC 9864, October 2025.
- [TR03116-3] Bundesamt für Sicherheit in der Informationstechnik. BSI TR-03116-3 – Kryptographische Vorgaben für Projekte der Bundesregierung, Teil 3: Intelligente Messsysteme, in der jeweils gültigen Fassung.

- [TR03111] Bundesamt für Sicherheit in der Informationstechnik. BSI TR-03111 – Elliptic Curve Cryptography, in der jeweils gültigen Fassung.
- [CP-SM-PKI] Bundesamt für Sicherheit in der Informationstechnik. Certificate Policy der Smart Metering PKI. Version 1.1.2, 25.01.2023.
- [RzÜ-API] BDEW. Regelungen zum Übertragungsweg für API-Webdienste. Version 1.2, Publikationsdatum 01.04.2025, anzuwenden ab 01.10.2025.
- [RzV] BDEW. Regelungen zum Verzeichnisdienst. Version 1.1, Publikationsdatum 01.04.2025, anzuwenden ab 01.10.2025.

Version	Datum	Änderung	Autor
0.9	28.09.2026	Initiale Erstellung	Arbeitsgruppe API